Manual Técnico de la Solución Base del Proyecto Oráculo: Búsqueda de Configuraciones de Prompt mediante Ascenso por Coordenadas Guiado por Trazas

Juan Felipe Henao

Universidad de Caldas

Análisis y Diseño de Algoritmos (ADAG1)

Luz Enith Guerrero Mendieta

5 de noviembre de 2026

Manual Técnico de la Solución Base del Proyecto Oráculo

Este manual describe cómo está construida la solución base del Proyecto Oráculo y cómo reproducir su ejecución sin ayuda de su autor. La solución busca una configuración de prompt (un índice por cada una de las ranuras *rol*, *estrategia*, *formato*, *estilo* y *cierre*, más la temperatura de generación) que maximice la precisión de un modelo de lenguaje sobre instancias con restricciones verificables, consultando el oráculo provisto por el curso (DanielMelo404, 2026). La configuración entregada es {rol: 0, estrategia: 0, formato: 0, estilo: 5, cierre: 1, temperatura: 0,0}, con una precisión de 14,0 % en las 100 instancias de búsqueda y de 10,3 % en las 87 instancias de validación.

El código se encuentra en un fork del repositorio del curso (Henao, 2026; JuManoel, 2026), en la rama `main`. Corresponde a la versión de la primera entrega, es decir, a la solución base del problema.

Entorno

La solución se ejecuta en Google Colab con una GPU NVIDIA T4. La Tabla 1 resume los componentes del entorno. La primera celda de cada notebook instala todas las dependencias; después de ejecutarla es obligatorio reiniciar la sesión, porque de lo contrario la librería `transformers` (Wolf et al., 2020) puede quedar cargada de forma incompleta.

Tabla 1

Componentes del Entorno de Ejecución

Elemento	Valor
Plataforma	Google Colab, entorno con GPU NVIDIA T4 (16 GB)
Lenguaje	Python 3 (versión de Colab; probado con 3.13)
Modelo	`Qwen/Qwen3-1.7B`, alias `qwen17b` (Qwen Team, 2025), en fp16, ≈ 3,4 GB de VRAM
Dependencias	`transformers`, `accelerate`, `bitsandbytes` ≥ 0.46.1, `nltk`, `spacy` (con `en_core_web_sm`), `emoji`, `langdetect`, `immutabledict`, `pandas`, `matplotlib`
Verificador	Repositorio `allenai/open-instruct` (Allen Institute for AI, s.f.)
Archivos del curso	`oraculo.py`, `ayudas.py` y `datos_visibles.json`, en la raíz del fork; la celda 1 descarga la misma versión para Colab
Almacenamiento opcional	Google Drive, para el caché del oráculo y el registro de evaluaciones
Pruebas sin GPU	Python 3.9 o superior, solo biblioteca estándar, en cualquier sistema operativo

Arquitectura

El repositorio es un fork de JuManoel/ADA-2026-2-GP01 (JuManoel, 2026). Los archivos del curso se conservan sin cambios en la raíz, y la solución se agrega en carpetas propias, como muestra la Figura 1.

Figura 1

Estructura del Repositorio

```
ADA-2026-2-GP01/  (fork)
├── README.md                      descripción de la solución
├── README_curso.md                README original del curso
├── oraculo.py                     del curso (caja negra, sin modificar)
├── ayudas.py                      del curso (sin modificar)
├── datos_visibles.json            benchmark del curso (sin modificar)
├── proyecto_oraculo.ipynb         notebook plantilla del curso (sin modificar)
├── entrega.json                   salida final: configuración ganadora
├── notebooks/
│   ├── notebook_busqueda.ipynb    búsqueda (punto 4) y evidencia
│   └── notebook_evaluacion.ipynb  evaluación de entrega.json
├── resultados/registro_busqueda.csv
├── pruebas/
│   ├── verificar_entrega.py
│   └── prueba_sintetica.py
└── docs/
    ├── manual_tecnico.pdf
    └── manual_tecnico.md
```

La Tabla 2 describe la responsabilidad de cada componente. El oráculo del curso se trata como una caja negra: se consulta, pero no se modifica.

Tabla 2

Responsabilidad de Cada Componente

Componente	Responsabilidad
`oraculo.py` (curso)	`Oraculo.evaluar(config, instancias, semilla)` genera y verifica las respuestas del modelo; `Oraculo.validar(config, n)` mide sobre la partición de validación; `espacio(temps)` lista las configuraciones válidas. Guarda un caché de respuestas.
`ayudas.py` (curso)	Carga el modelo y los datos, divide búsqueda y validación, y ofrece `ver_prompt`, `curva` y `entrega`.
`notebook_busqueda.ipynb`	Es `proyecto_oraculo.ipynb` con el punto 4 implementado: ascenso por coordenadas guiado por trazas, registro de cada evaluación, validación y exportación de `entrega.json`.
`notebook_evaluacion.ipynb`	Lee `entrega.json`, localiza los datos de prueba y reporta la precisión, sin editar código.
`registro_busqueda.csv`	Evidencia de las 68 evaluaciones de la búsqueda.
`verificar_entrega.py`	Comprueba que `entrega.json` sea válido y coincida con la mejor fila del registro.
`prueba_sintetica.py`	Ejecuta la misma lógica de búsqueda contra un oráculo sintético de resultado conocido.

Secciones del Notebook de Búsqueda

El notebook de búsqueda está organizado en 12 secciones, resumidas en la Tabla 3. Las secciones 1, 2 y 12 provienen de la plantilla del curso; las secciones 3 a 11 implementan y documentan el punto 4.

Tabla 3

Secciones del Notebook de Búsqueda

Sección	Función
1	Instala dependencias y descarga los archivos del curso.
2	Define los parámetros (`MODELO`, `SEMILLA`, `MAX_PASADAS`, `UMBRAL_TRAZA`, `PROBAR_TEMPERATURAS`, `USAR_DRIVE`, `GRUPO`, `SEMANA`) y carga el modelo, los datos y el oráculo con caché.
3	Detecta ranuras y opciones desde `espacio()`; `config_con` crea configuraciones vecinas que difieren en una sola ranura.
4	`evaluar_registrar`: evalúa, cuenta los fallos por familia de restricción y guarda una fila en el registro.
5	Evalúa la configuración inicial y mide el tiempo por evaluación.
6	Cuenta los fallos por familia y fija el orden de las ranuras con la hipótesis ranura → familias.
7	Ejecuta el ascenso por coordenadas con poda por trazas.
8	Prueba las temperaturas 0,3 y 0,7.
9	Muestra la tabla de evaluaciones, la curva de mejor precisión, el costo y la gráfica por ranura.
10	Presenta las opciones descartadas, el mapa observado ranura → familias y los fallos que persisten.
11	Ejecuta `oraculo.validar` sobre las 87 instancias de validación.
12	Escribe y descarga `entrega.json` y el registro.

Funcionamiento

Flujo del Programa

El programa sigue seis pasos. Primero carga el modelo, los datos (100 instancias de búsqueda y 87 de validación del benchmark del curso) y el oráculo. Luego evalúa la configuración inicial $c_0 = (0, 0, 0, 0, 0)$ con temperatura 0,0. En tercer lugar ordena las ranuras según los fallos que las trazas de c_0 asocian a cada una. Después repite pasadas de ascenso por coordenadas hasta la convergencia o hasta `MAX_PASADAS`. Finalmente ajusta la temperatura, valida la configuración y la exporta.

Algoritmo: Ascenso por Coordenadas Guiado por Trazas

El ascenso por coordenadas optimiza una variable a la vez manteniendo fijas las demás (Wright, 2015). En esta solución cada ranura del prompt es una coordenada, y las trazas de fallos del oráculo se usan para dos decisiones: el orden en que se recorren las ranuras y la poda de opciones. La Figura 2 presenta el pseudocódigo.

Figura 2

Pseudocódigo del Ascenso por Coordenadas Guiado por Trazas

```
actual ← c₀;  (p, F) ← evaluar(actual)            # F: fallos por familia
orden ← ranuras ordenadas por fallos asociados en F
descartadas[r] ← ∅ para cada ranura r
para pasada = 1 .. MAX_PASADAS:
    mejoró ← falso
    para r en orden:
        (p_base, F_base) ← (p, F);  mejor ← (actual[r], p, F)
        para cada opción o ≠ actual[r], o ∉ descartadas[r]:
            (p_o, F_o) ← evaluar(actual con r = o)
            si existe familia f con F_o[f] − F_base[f] ≥ UMBRAL_TRAZA
               y p_o ≤ p_base:
                descartadas[r] ← descartadas[r] ∪ {o}
            si p_o > mejor.p:  mejor ← (o, p_o, F_o)
        si mejor.o ≠ actual[r]:
            actual[r] ← mejor.o;  (p, F) ← (mejor.p, mejor.F)
            mejoró ← verdadero
    si no mejoró:  parar        # óptimo local respecto a una ranura
probar temperaturas 0,3 y 0,7 con las ranuras fijas
```

En caso de empate se conserva la opción actual, para no cambiar la configuración por diferencias atribuibles al ruido de muestreo.

Estructuras de Datos

La Tabla 4 resume las estructuras de datos del notebook de búsqueda. El acceso a las configuraciones oficiales se hace mediante un diccionario indexado por tuplas, lo que garantiza que nunca se evalúe una configuración inválida.

Tabla 4

Estructuras de Datos Principales

Estructura	Tipo	Contenido
Configuración	`dict`	Índices enteros de las cinco ranuras y la temperatura (`float`)
`INDICE`	`dict[tupla → config]`	Acceso en tiempo constante a la configuración oficial a partir de sus índices
`REGISTRO`	`list[dict]`	Una fila por evaluación; se exporta a CSV
`RESULTADOS`	`dict[tupla → (precisión, Counter)]`	Precisión y fallos por familia de cada configuración evaluada
`DESCARTADAS`	`dict[ranura → dict[opción → motivo]]`	Opciones podadas y la familia que motivó el descarte
Fallos por familia	`collections.Counter`	Familia (prefijo de `violo`, por ejemplo `startend`) → número de instancias

Condición de Parada y Costo

La búsqueda se detiene cuando una pasada completa no mejora la mejor precisión, o al alcanzar `MAX_PASADAS` = 3. Tomando como operación básica una consulta al modelo, con n instancias, m ranuras de k_p opciones, I pasadas y $|T|$ temperaturas, el costo máximo es:

$$T(m, k, n, I) \leq n\,[1 + I\,\Sigma_{p=1}^{m}\,(k_p - 1) + (|T| - 1)] = O(I \cdot m \cdot k \cdot n)$$

Con los valores del proyecto, la cota es $100 \cdot (1 + 3 \cdot 35 + 2) = 10.800$ consultas. El caché del oráculo y la poda reducen el costo real: la corrida evaluó 60 configuraciones distintas.

Uso

Evaluación de la Configuración Entregada

El notebook de evaluación no requiere editar código, rutas ni parámetros. Para ejecutarlo se siguen estos pasos:

1. Abrir `notebooks/notebook_evaluacion.ipynb` en Colab y seleccionar *T4 GPU*.
2. Ejecutar la celda 1 y luego *Entorno de ejecución ► Reiniciar sesión*.

3. De forma opcional, arrastrar al panel de archivos de Colab un archivo de instancias de prueba con el formato de `datos_visibles.json`.
4. Seleccionar *Entorno de ejecución ▸ Ejecutar todas*.

La entrada principal es `entrega.json`, que el notebook toma de la carpeta local o descarga del repositorio en GitHub. Su formato se muestra en la Figura 3. El archivo de instancias es una lista de objetos con, al menos, los campos `prompt` e `ids` (y `kwargs` para el verificador); las instancias marcadas con `"split": "descartada"` se ignoran.

Figura 3

Formato del Archivo de Entrada entrega.json

```
{"grupo": "G01",
 "config": {"rol": 0, "estrategia": 0, "formato": 0,
            "estilo": 5, "cierre": 1, "temperatura": 0.0},
 "semana": 5}
```

La Figura 4 muestra un ejemplo de salida. La precisión es la fracción de instancias en las que la respuesta cumplió todas sus restricciones (evaluación todo o nada). La lista de fallos indica la primera restricción violada en cada instancia fallida, agrupada por familia. El resultado se guarda en `resultado_evaluacion.json`.

Figura 4

Ejemplo de Salida del Notebook de Evaluación

```
Fuente:        partición de validación pública (demostración)
Instancias:    87
Configuración: {'rol': 0, 'estrategia': 0, 'formato': 0,
                'estilo': 5, 'cierre': 1} temperatura 0.0
PRECISIÓN:     10.3%
```

Reproducción de la Búsqueda

Para reproducir la búsqueda se abre `notebooks/notebook_busqueda.ipynb` en Colab con GPU T4, se ejecuta la celda 1 y se reinicia la sesión. Al ejecutar la celda 2 se debe aceptar el acceso a Google Drive marcando todas las casillas de permisos. Las secciones se ejecutan en

orden: cada evaluación nueva tarda alrededor de 205 s, y las que ya están en caché, 0 s. Al final se descargan `entrega.json` y `registro_busqueda.csv`.

Pruebas sin GPU

Desde la raíz del repositorio se ejecutan `python pruebas/verificar_entrega.py` y `python pruebas/prueba_sintetica.py`. Ninguna requiere GPU ni librerías externas.

Validación y Límites

Pruebas Disponibles

La Tabla 5 describe las pruebas incluidas en el repositorio y su resultado esperado. La prueba sintética usa un oráculo con ranuras independientes y semilla fija, por lo que su resultado es reproducible.

Tabla 5

Pruebas Disponibles y Resultado Esperado

Prueba	Qué verifica	Resultado esperado
`verificar_entrega.py`	Claves y tipos de `entrega.json`; índices entre 0 y 7; temperatura en {0,0; 0,3; 0,7}; coincidencia con la mejor fila del registro	Mensaje de verificación exitosa
`prueba_sintetica.py`	Con un oráculo sintético (semilla 7): la mejor precisión nunca baja, no se supera la cota $1 + I\,\Sigma(k_p - 1)$ y se encuentra el óptimo global	2 pasadas, 64 evaluaciones (cota: 106) y tres verificaciones aprobadas
Sección 11 del notebook de búsqueda	Generalización a 87 instancias no usadas en la búsqueda	10,3 %

Errores Conocidos y Soluciones

La Tabla 6 reúne los errores observados durante el desarrollo y la forma de resolverlos.

Tabla 6

Errores Conocidos y Soluciones

Síntoma	Causa	Solución
`MessageError: credential propagation was unsuccessful`	Permisos de Drive no concedidos o ventana emergente bloqueada	Repetir marcando todas las casillas, montar Drive desde el panel de archivos o usar `USAR_DRIVE = False`
`cargar_modelo` falla con un error poco claro	No se reinició la sesión tras la celda 1	Reiniciar la sesión y continuar desde la celda 2
Mensaje «No hay GPU»	El entorno no tiene GPU asignada	Seleccionar T4 GPU y empezar desde la celda 1
La búsqueda se interrumpe	Desconexión o límite de GPU de Colab	Volver a ejecutar: con el caché en Drive retoma sin recalcular

Supuestos

La solución supone que cada consulta tiene el mismo costo y que su resultado es binario. Todas las configuraciones se comparan sobre las mismas 100 instancias de búsqueda, que provienen del mismo conjunto de datos usado por GEPA (Agrawal et al., 2025) y por Chang y Chen (2026). El orden inicial de las ranuras se basa en una hipótesis ranura → familias tomada de la documentación del curso, que las trazas confirmaron solo en parte.

Limitaciones y Componentes Pendientes

El resultado es un óptimo local: ningún cambio de una sola ranura lo mejora, pero el techo reportado por el curso es cercano al 24 % (DanielMelo404, 2026). La poda es permanente y su umbral de tres fallos es bajo frente al ruido de 100 instancias; además, la diferencia entre 14 % y 12 % equivale a dos instancias y está dentro del error de muestreo.

Quedan pendientes para las siguientes entregas el algoritmo exacto de referencia (backtracking con poda), Successive Halving sobre instancias sintéticas de tamaño creciente, la comparación entre reparto uniforme y adaptativo del presupuesto, y la validación experimental del crecimiento de $T(n)$ frente a n.

Referencias

Agrawal, L. A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Dimakis, A. G., Stoica, I., Klein, D., Zaharia, M., y Khattab, O. (2025). *GEPA: Reflective prompt evolution can outperform reinforcement learning* (arXiv:2507.19457). arXiv. https://arxiv.org/abs/2507.19457

Allen Institute for AI. (s.f.). *open-instruct* [Software]. GitHub. https://github.com/allenai/open-instruct

Chang, Y., y Chen, X. (2026). *Naive prompt optimization: Rethinking the need for complex prompt search* (arXiv:2608.27266). arXiv. https://arxiv.org/abs/2608.27266

DanielMelo404. (2026). *Proyecto-Oraculo-AyD* [Software]. GitHub. https://github.com/DanielMelo404/Proyecto-Oraculo-AyD

Henao, J. F. (2026). *ADA-2026-2-GP01: Solución base del Proyecto Oráculo* [Software]. GitHub. https://github.com/Pipe-1z/ADA-2026-2-GP01

JuManoel. (2026). *ADA-2026-2-GP01* [Software]. GitHub. https://github.com/JuManoel/ADA-2026-2-GP01

Qwen Team. (2025). *Qwen3 technical report* (arXiv:2505.09388). arXiv. https://arxiv.org/abs/2505.09388

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., … Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. En *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Wright, S. J. (2015). Coordinate descent algorithms. *Mathematical Programming, 151*(1), 3–34. https://doi.org/10.1007/s10107-015-0892-3